PRACTICAL – 9

Implement Prim's Algorithm

Objective

To implement Prim's Algorithm to find the Minimum Spanning Tree (MST) of a connected, undirected, weighted graph.

Algorithm

i. **Start the program and read the number of vertices and the adjacency matrix.**

ii. **Initialize all vertices as not visited and set the key values to infinity.**

iii. **Set the key value of the starting vertex to 0.**

iv. **Select the unvisited vertex with the minimum key value.**

v. **Mark the selected vertex as visited.**

vi. **Update the key values of its adjacent unvisited vertices if a smaller edge weight is found.**

vii. **Repeat until all vertices are visited.**

viii. **Display the edges of the Minimum Spanning Tree and their total cost.**

ix. **Stop.**

Code

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

int main() {
   int n;
   cout << "Enter number of vertices: ";
```

```
cin >> n;

vector<vector<int>> graph(n, vector<int>(n));
cout << "Enter adjacency matrix (0 for no edge):\n";

for (int i = 0; i < n; i++) {
   for (int j = 0; j < n; j++) {
      cin >> graph[i][j];
   }
}

vector<int> key(n, INT_MAX);
vector<int> parent(n, -1);
vector<bool> visited(n, false);

key[0] = 0;
int totalCost = 0;

for (int count = 0; count < n; count++) {
   int u = -1;

   for (int i = 0; i < n; i++) {
      if (!visited[i] &&
         (u == -1 || key[i] < key[u])) {
         u = i;
      }
   }

   if (u == -1 || key[u] == INT_MAX) {
      cout << "Graph is not connected.\n";
      return 0;
   }

   visited[u] = true;

   for (int v = 0; v < n; v++) {
      if (graph[u][v] != 0 &&
         !visited[v] &&
         graph[u][v] < key[v]) {
         key[v] = graph[u][v];
         parent[v] = u;
      }
   }
}
```

```
    cout << "\nEdges in Minimum Spanning Tree:\n";

    for (int i = 1; i < n; i++) {
        cout << parent[i] << " - " << i
             << " : " << key[i] << "\n";
        totalCost += key[i];
    }

    cout << "Total cost of MST: " << totalCost << "\n";

    return 0;
}
```

Output:

```
Output

Enter number of vertices: 4
Enter adjacency matrix (0 for no edge):
0 2 0 6
2 0 3 8
0 3 0 1
6 8 1 0

Edges in Minimum Spanning Tree:
0 - 1 : 2
1 - 2 : 3
2 - 3 : 1
Total cost of MST: 6
```

Time Complexity:

Time Complexity: $O(V^2)$
Space Complexity: $O(V^2)$
Where V is the number of vertices in the graph.

Conclusion:

Thus, Prim's Algorithm was successfully implemented in C++ to find the Minimum Spanning Tree of a connected, weighted graph with minimum total edge cost.
Important: Enter 0 when there is no edge between two vertices. This program uses vertices numbered from 0 to n-1.